

<<HELLO CS!

UNC CS OPEN HOUSE



- CLUB SHOWCASE
 - CAREER KICK-OFF
 - UNC CS ADVISING
 - UNC CS UNDERGRADUATE RESEARCH
- + **COOKIES POPSICLES!**



NetApp™



AUG 25
6- 8 PM

SITTERSON HALL



College of Arts
and Sciences

Computer Science

Classroom Habits

- Place bags under chairs so that it is easy to walk around.
- Our goal is to learn! And the first step is not being distracted!
 - Phones
 - Place your phone on Do Not Disturb so that it does not distract from your learning.
 - Phones should be placed in your bags during class.
 - Laptops
 - Should only be out if you are typing notes
- We are so fortunate to be learning in this room. At the end of class please
 - Clean white boards
 - Place white boards back where they belong
 - Tuck in your chair
 - Clean up trash



Intro to C

COMP211 Connor McMahon

Compiling and Assembling

Machine Code

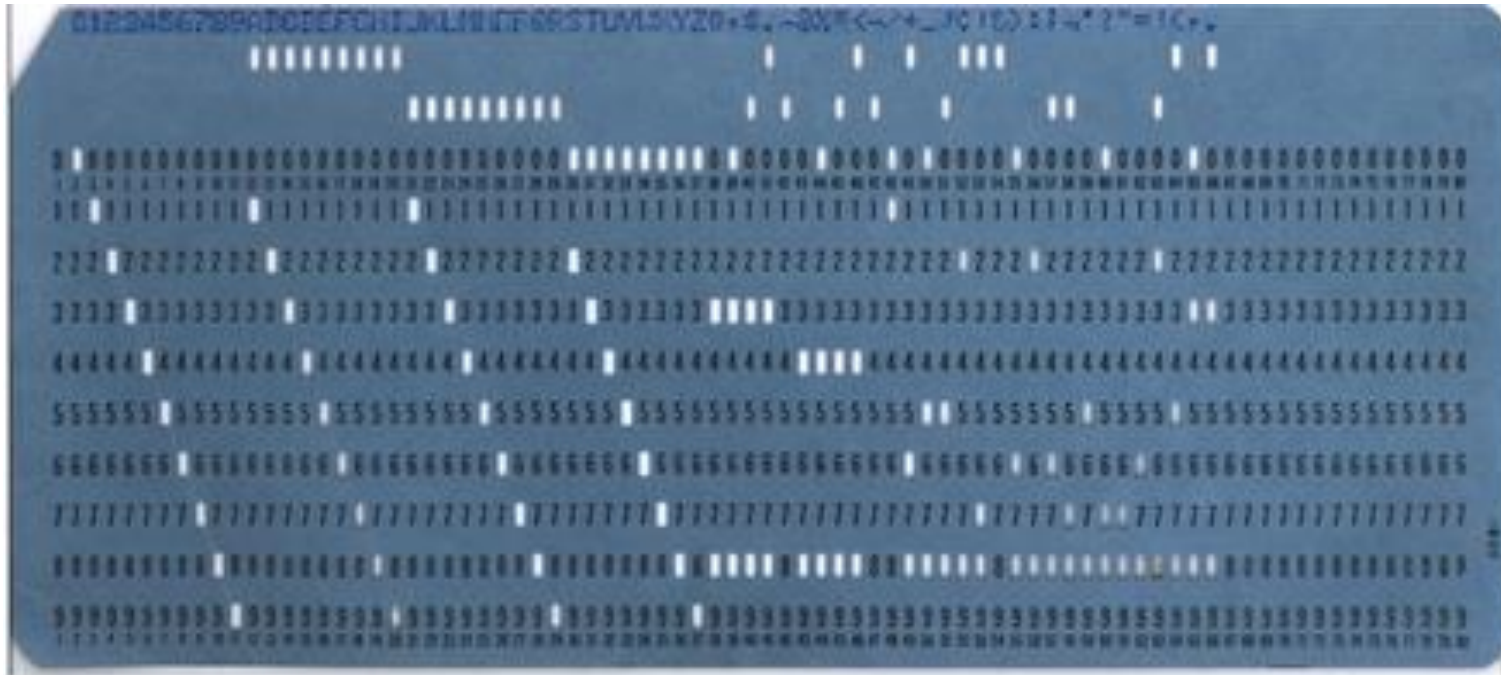
- Computers cannot directly understand the programs that we write (Python, Java, etc). Instead, they understand something called machine code.
- **Machine code** consists of binary instructions (strings of 1s and 0s) that tell the computer what operations to perform.

Machine code

```
0010011110111101111111111111000
1010111110111110000000000000100
00000011101000001111000000100101
1010111111000100000000000001000
1010111111000010000000000001000
0000000000000000000000000000000
00000000010000100000000000011000
0000000000000000000100000010010
00000011110000001110100000100101
10001111101111100000000000000100
001001111011111010000000000001000
0000001111100000000000000001000
0000000000000000000000000000000
```

A Brief History of Programming

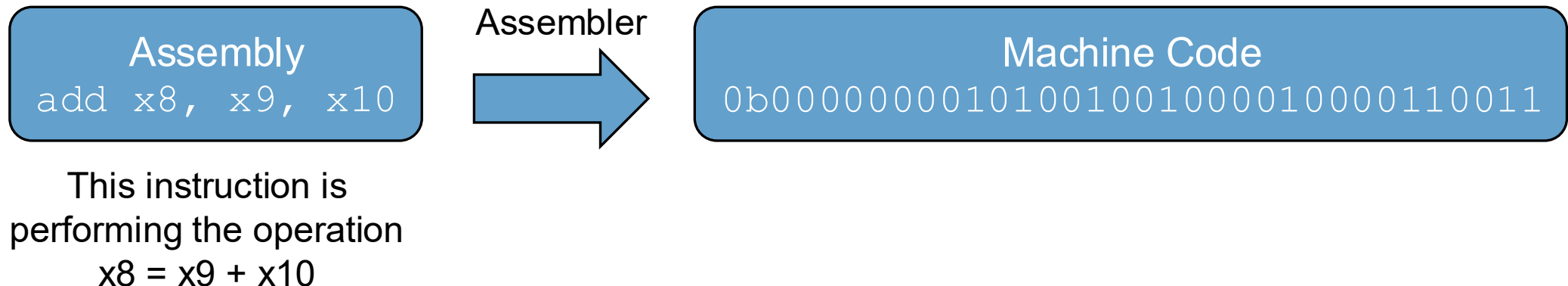
- Before the 1950s, programming was done entirely in **machine code**
- These instructions were entered manually using switches, punch cards, or paper tape.



IBM 80-Column Punch Card

A Brief History of Programming

- In the early 1950s, **assembly language** was introduced as a more human-readable way to write programs.
- Machines cannot read assembly language programs (they can only read machine code)!
- **Assemblers** (programs that translate assembly code into machine code) were developed to automate the conversion process.
- There are lots of different assembly languages. You'll learn more about them in COMP311!



A Brief History of Programming

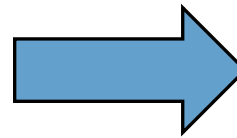
- With the invention of assembly language, programmers could write and debug code much more efficiently!
- Assembly language is known as a **low-level language**

Assembly Language (Low-level language)

```
1 square(int):  
2     addiu    $sp,$sp,-8  
3     sw      $fp,4($sp)  
4     move    $fp,$sp  
5     sw      $4,8($fp)  
6     lw      $2,8($fp)  
7     nop  
8     mult    $2,$2  
9     mflo    $2  
10    move    $sp,$fp  
11    lw      $fp,4($sp)  
12    addiu    $sp,$sp,8  
13    jr      $31  
14    nop
```

Human readable

Assembler



Machine code

```
0010011110111101111111111111111000  
101011111011111000000000000000100  
00000011101000001111000000100101  
10101111110001000000000000001000  
10101111110000100000000000001000  
00000000000000000000000000000000  
000000000100001000000000000011000  
000000000000000000000100000010010  
00000011110000001110100000100101  
10001111101111100000000000000100  
001001111011111010000000000001000  
00000011111000000000000000001000  
00000000000000000000000000000000
```

Machine-readable

A Brief History of Programming

- In the late 1950s, **high-level languages** began abstracting away the details of hardware, focusing on logic and algorithms.
- Key Early Languages:
 - 1957: FORTRAN (FORmula TRANslation):
 - First widely adopted high-level language.
 - Developed by IBM for scientific and engineering calculations.
 - 1958: LISP (LISt Processor):
 - Pioneered functional programming and symbolic processing.
 - 1959: COBOL (COmmon Business-Oriented Language):
 - Designed for business applications.
 - Emphasized readability and portability.
 - Became a standard for enterprise and financial systems.

High-Level Languages

- Most programming languages you've heard of are **high-level programming languages** (Python, Java, C, C++).
- C is a **compiled high-level programming language**.
- To execute a compiled high-level programming language, the program is first translated into assembly language by a **compiler**, then an **assembler** translates the assembly language into machine code.
- **Interpreted languages** like Python are executed differently. We'll talk more about interpreted languages later.

Assembling and Compiling

C
(High-level language)

```
1 int square(int num) {  
2     return num * num;  
3 }
```

Easy for humans to
read/write

Compiler


Assembly Language
(Low-level language)

```
1 square(int):  
2     addiu    $sp,$sp,-8  
3     sw       $fp,4($sp)  
4     move     $fp,$sp  
5     sw       $4,8($fp)  
6     lw       $2,8($fp)  
7     nop  
8     mult     $2,$2  
9     mflo     $2  
10    move     $sp,$fp  
11    lw       $fp,4($sp)  
12    addiu    $sp,$sp,8  
13    jr       $31  
14    nop
```

Human readable

Assembler


Machine code

```
00100111101111011111111111111111000  
1010111110111110000000000000000100  
00000011101000001111000000100101  
101011111100010000000000000001000  
101011111100001000000000000001000  
00000000000000000000000000000000  
00000000010000100000000000011000  
000000000000000000001000000010010  
00000011110000001110100000100101  
10001111101111100000000000000100  
001001111011111010000000000001000  
0000001111100000000000000001000  
00000000000000000000000000000000
```

Machine-readable

Key Terms

- Machine Code
- Assembly Language
- Low-level language
- High-level language
- Compiler
- Assembler
- Compiled language
- FORTRAN, LISP, COBOL

C Introduction

Your First C Program

All C programs begin executing at the main function.

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

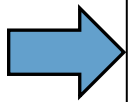
- `main` returns an integer value that is used to indicate the exit status of the program.
 - A successful execution returns 0.
 - An abnormal execution returns a non-zero value.

Your First C Program

To use printf, you
must explicitly
include `stdio.h`.



printf is a
standard
library
function that
prints output.



```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

Standard Library

- The **C Standard Library** contains a collection pre-written functions that handle common tasks like input/output, string manipulation, mathematical operations, and memory management.
- Library functions are grouped together in **header files**.
- C does not automatically include any standard library functions.

Header Files

- A header file contains the declarations of the functions, macros, and types you can use.
 - **Function Declaration:** the function's name, return type, and parameters
 - **Function Definition:** the body of the function

Header File	Purpose	Examples of Functions
<code><stdio.h></code>	Input/output functions	<code>printf, getchar, putchar, fopen</code>
<code><stdlib.h></code>	General utilities	<code>malloc, free, exit, atoi, rand</code>
<code><string.h></code>	String manipulation	<code>strlen, strcpy, strcat, strcmp</code>
<code><math.h></code>	Mathematical operations	<code>sqrt, pow, sin, cos, tan, exp</code>
<code><time.h></code>	Date and time utilities	<code>time, clock, localtime, gmtime</code>

Compiling, Assembling, and Running your First C Program

1. Write your program.
2. Save the file with a **.c** extension. Say **hello.c**.
3. Open a terminal and navigate to the directory where your program is located.
4. Compile and assemble the program

```
$ gcc hello.c
```

gcc hello.c converts your human-readable C code into a machine-readable **executable file**.

Compiling and Assembling your Program

- **gcc hello.c**
 - **gcc** : invokes the GNU C Compiler
 - **hello.c** is the source file you want to compile
 - By default, this produces an executable file named **a.out**.
 - Use the **-o** flag to specify a name for the executable. The executable name must immediately follow **-o**.

Command	Executable Name
gcc hello.c	a.out
gcc hello.c -o hello	hello
gcc -o hello hello.c	hello
gcc hello.c -o hi	hi

Compiling, Assembling, and Running your First C Program

1. Write your program.
2. Save the file with a .c extension.
 - For example, hello.c
3. Open a terminal and navigate to the directory where your program is located.
4. Compile and assemble the program

```
$ gcc hello.c
```

5. Run the program

```
$ ./a.out
```

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

hello.c

Running your Program

- **./a.out**
 - **.** = current directory
 - **./a.out** → run **a.out** from current directory
- If you named your executable something else
 - For example, if you named it **hello**
 - Run with **./hello**
- *(Note: **./** is required because the current directory is not in your PATH.)*

Your turn!

- [Multiple select] Which of the following set of commands can you use to compile, assemble, and execute the program in a file named `my_program.c`

A

```
$ gcc my_program.c a.out  
$ ./my_program.c
```

B

```
$ gcc my_program.c -o my_program  
$ ./a.out
```

C

```
$ gcc my_program.c -o my_prog  
$ ./my_prog
```

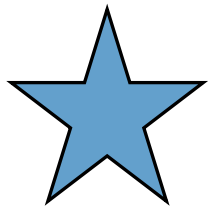
D

```
$ gcc my_program.c  
$ ./a.out
```

E

```
$ gcc my_program.c -o my_program  
$ ./my_program.c
```

Your turn!



- [Multiple select] Which of the following set of commands can you use to compile, assemble, and execute the program in a file named `my_program.c`

A

```
$ gcc my_program.c  
$ ./my_program.c
```

B

```
$ gcc my_program.c -o my_program  
$ ./a.out
```

C

```
$ gcc my_program.c -o my_prog  
$ ./my_prog
```

D

```
$ gcc my_program.c  
$ ./a.out
```

E

```
$ gcc my_program.c -o my_program  
$ ./my_program.c
```

Key Terms

- Standard Library
- main function
- gcc
- GNU
- Executable

Variables in C

Variables in C

- **Scope**

- Where in a program you can use a variable

- **Lifetime**

- When the program exists in memory

- **Type**

- The range of values the variable can represent
- How the value will be interpreted

C Numeric Types

Type name	Usual size	Values stored	How to declare
char	1 byte	integers	char x;
short	2 bytes	signed integers	short x;
int	4 bytes	signed integers	int x;
long	4 or 8 bytes	signed integers	long x;
long long	8 bytes	signed integers	long long x;
float	4 bytes	signed real numbers	float x;
double	8 bytes	signed real numbers	double x;

8 bits = 1 byte

Type Sizes

- The exact number of bytes for each of the C types might vary from one architecture to the next.
- You can print the exact size on a given machine using C's `sizeof` operator, which takes the name of a type as an argument and evaluates to the number of bytes used to store that type.

printf

- C's **printf** function is very similar to the **System.out.printf** method in Java, where the caller specifies a format string to print.
- The format string often contains formatting specifiers, such as **special characters** that will print tabs (**\t**) or newlines (**\n**), or placeholders for values in the output.
- Placeholders consist of **%** followed by a type specifier letter.
- For each placeholder in the format string, **printf** expects an additional argument.

Common Special Characters

- A **special character** is a character that has a specific meaning or function in a programming language rather than simply representing ordinary text.
- In C, **escape sequences** use a backslash (\) to represent special characters
- These allow programmers to include characters or actions in a string that may be difficult to type or represent directly.

Special Character	Name	Meaning / Use	Example
\n	Newline	Moves to the next line	"Hello\nWorld"
\t	Tab	Inserts a horizontal tab	"Name:\tJohn"
\\	Backslash	Prints a \	"C:\\Users"
\"	Double quote	Prints " inside a string	"He said \"Hello\""
\0	Null character	Marks the end of a C string	"Hello\0"
%%	Percent sign	Prints % when using printf	printf("50%%");

Common Placeholders

Placeholder	Used for	Example	Output
%d	Signed integer	<code>printf("%d", 25);</code> 25	25
%u	Unsigned integer	<code>printf("%u", 25);</code>	25
%f	Floating-point number	<code>printf("%f", 3.14);</code>	3.14
%c	Single character	<code>printf("%c", 'A');</code>	A
%s	String	<code>printf("%s", "Hello");</code>	Hello

Type Sizes

- The exact number of bytes for each of the C types might vary from one architecture to the next. You can print the exact size on a given machine using C's **sizeof** operator, which takes the name of a type as an argument and evaluates to the number of bytes used to store that type. For example:

```
printf("number of bytes in an int: %lu\n", sizeof(int));  
printf("number of bytes in a short: %lu\n", sizeof(short));
```

- The **sizeof** operator evaluates to an unsigned long value, so in the call to **printf**, use the placeholder **%lu** to print its value.

The char Type

- The C char type stores a numeric value. However, it's often used by programmers to store the value of an **ASCII character**.
- With 8 bits, an unsigned char can hold the values $[0, 2^8-1] = [0, 255]$

Characters

ASCII

- **ASCII** stands for American Standard Code for Information Interchange.
- ASCII is a code that lets computers represent text using numbers.

Character	ASCII value
A	65
B	66
a	97
0	48
Space	32
!	33

- So when a computer encounters the ASCII number **65**, it interprets it as the letter **A**.

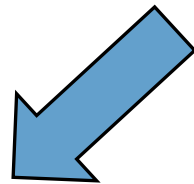
Full ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

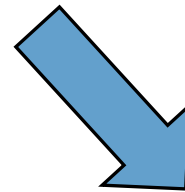
Bits are Just Bits!

- To give bits meaning, you need to assign them a representation, which defines how they correspond to meaningful information, such as numbers or text.

Let's say I have a char holding the value
0b 0100 0001



If I interpret this value as a
decimal integer, then I
would read it as 65.



If I interpret this value as an
ASCII character, then I
would read it as 'A'.

Characters

```
#include <stdio.h>

int main() {
    printf("%c\n", 'A');
}
```

```
learncli$ gcc char_ex0.c
learncli$ ./a.out
A
```

%c prints the character

Characters

```
#include <stdio.h>

int main() {
    printf("%d\n", 'A');
}
```

```
learncli$ gcc char_ex1.c
learncli$ ./a.out
65
```

%d prints a decimal integer.

In this case, it prints the character code that corresponds to `A`.

Characters

```
#include <stdio.h>

int main() {
    printf("%c\n", 67);
}
```

```
learncli$ gcc char_ex2.c
learncli$ ./a.out
C
```

%c prints the character whose ASCII
numeric representation is 67

Characters

```
#include <stdio.h>

int main() {
    printf("%d\n", 87);
}
```

```
learncli$ gcc char_ex3.c
learncli$ ./a.out
87
```

%d prints a decimal integer.

Full ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Characters

What does this program print?

```
#include <stdio.h>

int main() {
    printf("%d\n", 'a' - 'C');
}
```

Characters

```
#include <stdio.h>

int main() {
    printf("%d\n", 'a' - 'C');
}
```

The numeric ASCII representation of C is 67.

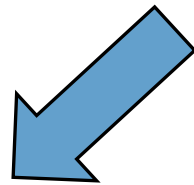
The numeric ASCII representation of a is 97.

$$97 - 67 = 30$$

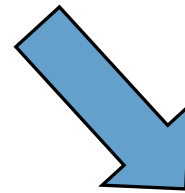
Bits are Just Bits!

- To give bits meaning, you need to assign them a representation, which defines how they correspond to meaningful information, such as numbers or text.

Let's say I have a variable holding the value
0b 0100 0001



If I interpret this value as a **decimal integer**, then I would read it as 65.



If I interpret this value as an **ASCII character**, then I would read it as 'A'.

Why We Are Learning This

- ASCII is not specific to the C programming language.
- Computers ultimately represent all data using binary, including letters, digits, and symbols.
- Regardless of the programming language you use, characters must be represented numerically so that the computer can store and process them.
- Learning ASCII helps us understand this fundamental relationship between characters and their numeric representations.

Full ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Reading Characters from Standard Input

We need to include `stdio.h` so
that we can use the functions
`getchar()` and `printf()`



`getchar()` reads one
character from the
input (like a keyboard
or a file) and returns it
as an **integer**. →

```
#include <stdio.h>

int main() {
    int my_char = getchar();
    printf("The user entered: %c\n", my_char);
}
```

Determining if a Character is a Letter

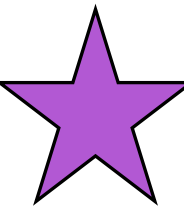
The following program determines if the user has entered any letter.
Fill in the blank.

```
#include <stdio.h>

int main() {
    int my_char = getchar();
    if (_____) {
        printf("The user entered a letter.\n");
    } else {
        printf("The user did not enter a letter.\n");
    }
}
```

- A. `my_char >= 'A' && my_char <= 'z'`
- B. `my_char >= 'A' || my_char <= 'z'`
- C. `my_char >= 'A' && my_char <= 'Z' || my_char >= 'a' && my_char <= 'z'`
- D. `my_char >= 'A' && my_char <= 'Z' && my_char >= 'a' && my_char <= 'z'`

Determining if a Character is a Letter



The following program determines if the user has entered any letter.
Fill in the blank.

```
#include <stdio.h>

int main() {
    int my_char = getchar();
    if (_____) {
        printf("The user entered a letter.\n");
    } else {
        printf("The user did not enter a letter.\n");
    }
}
```

Note, the C standard library includes the function `isalpha()` which we could have used to check if the character was an alphabet. It is defined in `<ctype.h>`.

- A. `my_char >= 'A' && my_char <= 'z'`
- B. `my_char >= 'A' || my_char <= 'z'`
- C. `(my_char >= 'A' && my_char <= 'Z') || (my_char >= 'a' && my_char <= 'z')`
- D. `(my_char >= 'A' && my_char <= 'Z') && (my_char >= 'a' && my_char <= 'z')`

End-of-File

This program is a simple echo program. It reads characters from the standard input until it encounters the **end-of-file (EOF)** marker, and it outputs each character as it reads it. Essentially, it copies input to output, one character at a time.

putchar() takes the character stored in c and prints it to the output. →

```
#include <stdio.h>

int main() {
    int c = getchar();
    while (c != EOF) {
        putchar(c);
        c = getchar();
    }
}
```

← EOF stands for **End of File**. It is a special value returned by getchar() when there's no more input to read.

You signal EOF by pressing **Ctrl+D** (Linux/macOS) or **Ctrl+Z** (Windows) when entering input manually.

End-of-File

- EOF stands for **End of File**. It is a special value returned by `getchar()` when there's no more input to read.
- EOF is the reason that `getchar()` returns an `int` instead of a `char`
- EOF must be distinguishable from any valid character
 - In the ASCII character set, all valid characters are encoded as integers from 0 to 255.
 - If EOF were encoded as a `char`, it could potentially collide with valid character values.
 - For example, if EOF were represented as 255 on a system where `char` is unsigned, it would conflict with the ASCII value for the `ÿ` character.

Characters

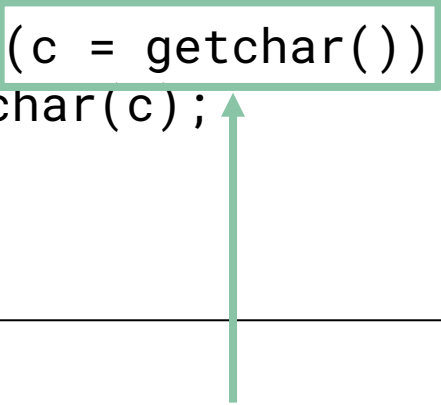
The program on the right is a more concise version of the program on the left.

```
#include <stdio.h>

int main() {
    int c = getchar();
    while (c != EOF) {
        putchar(c);
        c = getchar();
    }
}
```

```
#include <stdio.h>

int main() {
    int c;
    while ((c = getchar()) != EOF) {
        putchar(c);
    }
}
```



First, C gets the next char.
Then the while checks if that char is EOF.

Characters

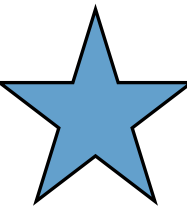
- Given the input, “HeLIO, WorLD!” what would the program’s output be?

```
#include <stdio.h>

int main() {
    int c;
    while ((c = getchar()) != EOF) {
        if (c >= 'a' && c <= 'z') {
            c = c - ('a' - 'A');
        }
        putchar(c);
    }
}
```

- A. hello, world!
- B. HLO, WLD
- C. HELLO, WORLD!
- D. e1, or!

Characters



- Given the following input, what would the program's output be? HeLIO, WorLD!

```
#include <stdio.h>

int main() {
    int c;
    while ((c = getchar()) != EOF) {
        if (c >= 'a' && c <= 'z') {
            c = c - ('a' - 'A');
        }
        putchar(c);
    }
}
```

- A. hello, world!
- B. HLO, WLD
- C. HELLO, WORLD!
- D. e1, or!